In [2]:
```
import os
os.environ["OMP_NUM_THREADS"] = "1"

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_wine
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    silhouette_score
)

# Load Wine dataset
wine = load_wine()

X = wine.data
y = wine.target   # True labels (only for evaluation)

print("Feature names:", wine.feature_names)
print("Target names:", wine.target_names)

# ------------------------------
# Feature Scaling
# ------------------------------
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ------------------------------
# K-Means Clustering
# ------------------------------
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

clusters = kmeans.fit_predict(X_scaled)

# Cluster centroids
centroids = kmeans.cluster_centers_

# ------------------------------
# PCA for Visualization
# ------------------------------
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
centroids_pca = pca.transform(centroids)
```

```
# ----------------------------
# Plot Clusters
# ----------------------------
plt.figure(figsize=(8, 6))

colors = ['red', 'green', 'blue']

for i in range(3):
    plt.scatter(
        X_pca[clusters == i, 0],
        X_pca[clusters == i, 1],
        s=50,
        color=colors[i],
        label=f"Cluster {i}"
    )

# Plot centroids
plt.scatter(
    centroids_pca[:, 0],
    centroids_pca[:, 1],
    s=200,
    c='yellow',
    marker='X',
    edgecolor='black',
    label='Centroids'
)

plt.title("K-Means Clustering on Wine Dataset (PCA View)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.grid()
plt.show()

# ======================================================
# CONFUSION MATRIX + PERFORMANCE METRICS
# ======================================================

# NOTE:
# KMeans cluster labels are arbitrary.
# We map cluster labels to true labels manually.

from scipy.stats import mode

labels = np.zeros_like(clusters)

for i in range(3):
    mask = (clusters == i)
    labels[mask] = mode(y[mask], keepdims=True).mode[0]

# ----------------------------
# Confusion Matrix
# ----------------------------
cm = confusion_matrix(y, labels)
```

```
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=wine.target_names,
    yticklabels=wine.target_names
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# ----------------------------
# Performance Metrics
# ----------------------------
accuracy = accuracy_score(y, labels)

precision = precision_score(
    y,
    labels,
    average='weighted'
)

recall = recall_score(
    y,
    labels,
    average='weighted'
)

f1 = f1_score(
    y,
    labels,
    average='weighted'
)

silhouette = silhouette_score(X_scaled, clusters)

print("\n===== PERFORMANCE METRICS =====")

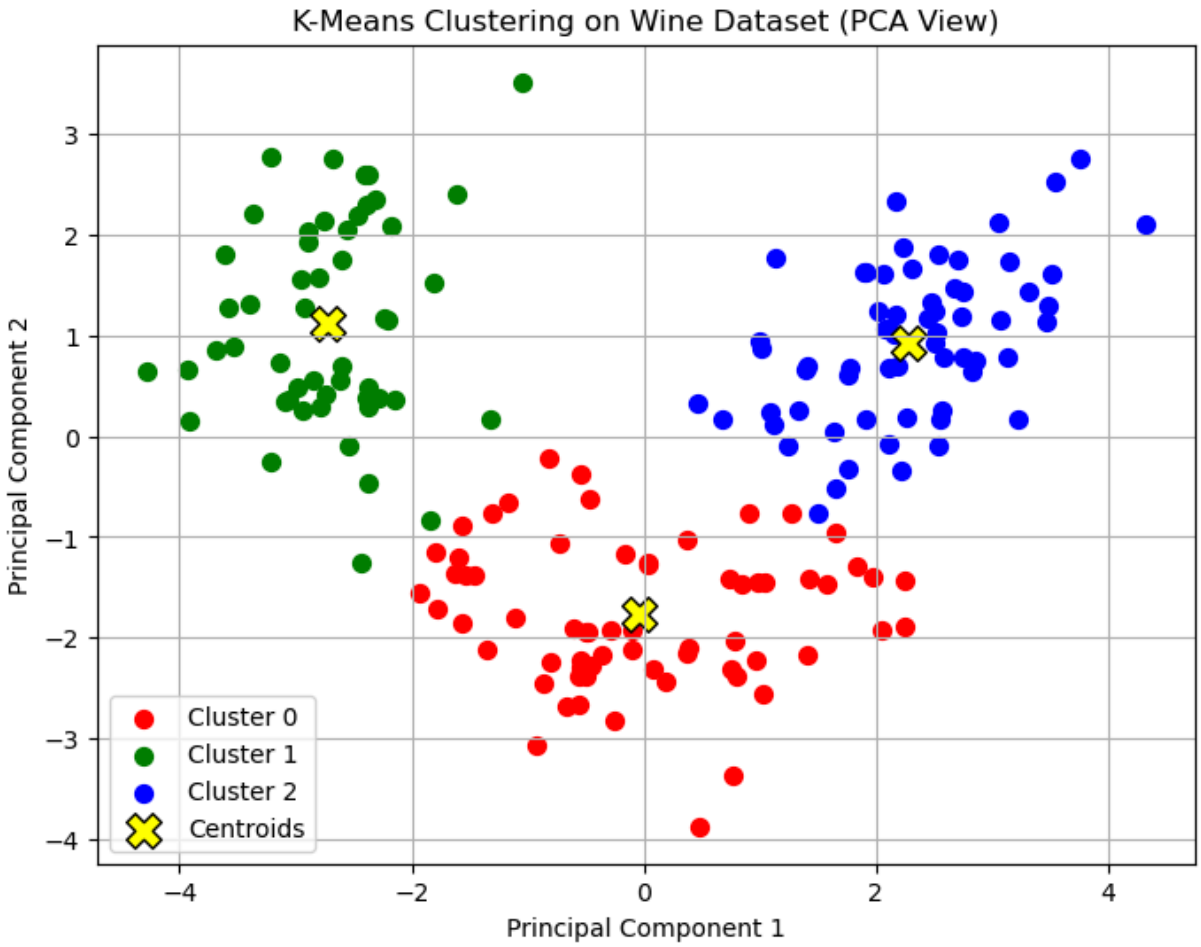
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1 Score  : {f1:.4f}")
print(f"Silhouette Score : {silhouette:.4f}")

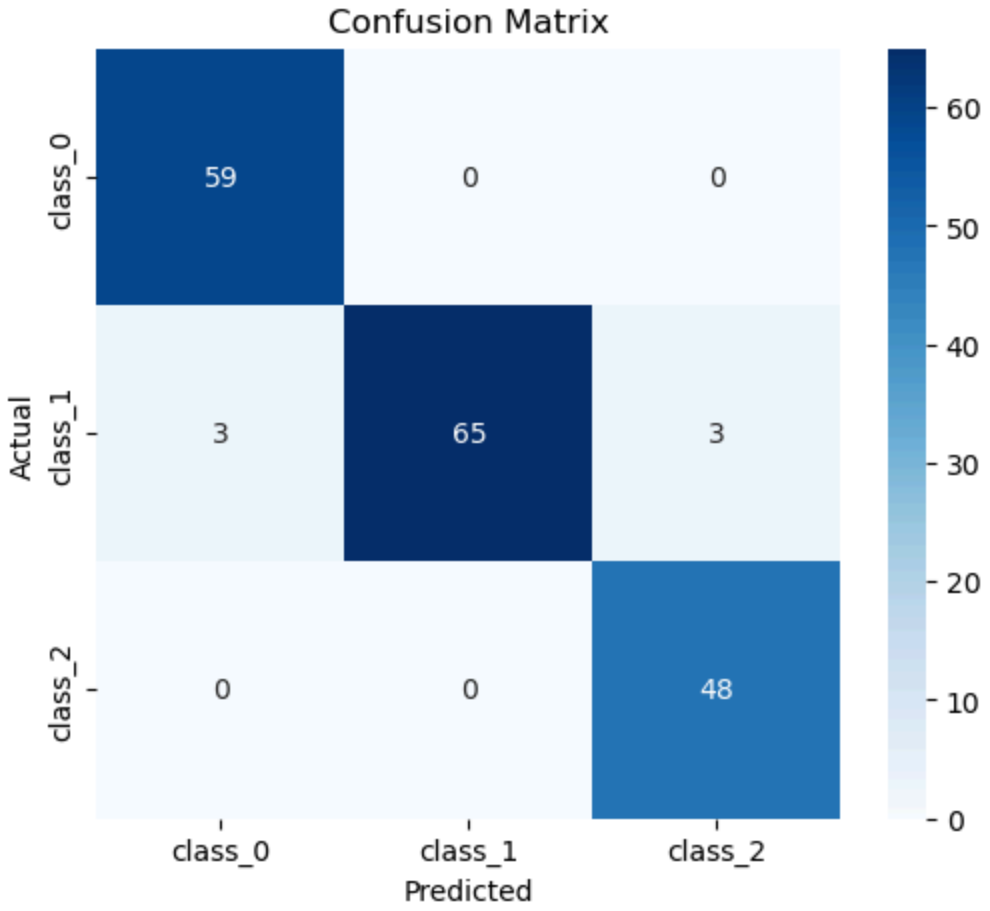
# ----------------------------
# Classification Report
# ----------------------------
print("\n===== CLASSIFICATION REPORT =====")

print(
    classification_report(
```

```
        y,
        labels,
        target_names=wine.target_names
    )
)
```

```
Feature names: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 't
otal_phenols', 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_inten
sity', 'hue', 'od280/od315_of_diluted_wines', 'proline']
Target names: ['class_0' 'class_1' 'class_2']
```






```
===== PERFORMANCE METRICS =====
Accuracy  : 0.9663
Precision : 0.9681
Recall    : 0.9663
F1 Score  : 0.9660
Silhouette Score : 0.2849

===== CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

     class_0       0.95      1.00      0.98        59
     class_1       1.00      0.92      0.96        71
     class_2       0.94      1.00      0.97        48

    accuracy                           0.97       178
   macro avg       0.96      0.97      0.97       178
weighted avg       0.97      0.97      0.97       178
```

In [ ]: